TICKET TEXT CONSTRUCTION BENCHMARK

Assignment 2 - CO2 - Task 1 | Technical experiment report

GitHub repository link: [Insert the working repository URL before submission]

Key finding: for the two larger workloads, repeated String concatenation was substantially slower than the mutable-buffer approaches. For a single short ticket, the measured differences were very small.

1. Objective and problem context

A railway e-ticket combines fields such as PNR, passenger, train, coach, berth and fare into one printable text block. The experiment compares three ways to create that text: repeated concatenation with String, append operations with StringBuilder, and append operations with StringBuffer. The aim is to determine whether the choice matters for a small ticket and under larger ticket-generation loads.

2. Prediction before implementation

String is immutable. An operation that concatenates a new part produces a new String containing the accumulated text and the new part. In a loop, earlier accumulated values become obsolete, so repeated concatenation creates many temporary String objects and recopies the growing prefix. The cost should increase sharply as the number of fields and tickets grows.

StringBuilder and StringBuffer are mutable. Their append methods add characters to a reusable internal buffer, avoiding a new String for every field. A final String is created when toString() is called. StringBuilder is not synchronized; StringBuffer synchronizes its methods. Therefore, both are predicted to outperform repeated String concatenation for large workloads, while the difference for one short ticket should be negligible compared with measurement noise.

3. Workload design

Workload	Tickets	Fields per ticket	Reason
Short ticket	1	6	Small baseline workload
High load	10,000	50	Many ticket generations
Wide ticket	1,000	200	Additional workload with many fields

The ticket parts are prepared before timing and reused across the tickets in a workload. This isolates the text-construction operation from input generation. Each approach receives identical parts and produces the same sample text. The program checks output equality before starting the measurements.

4. Implementation and measurement method

The supplied Java source file is TicketBenchmark.java. It implements three TicketBuilder strategies. For each ticket, the String strategy repeatedly applies += to the accumulated result. The other two strategies create one buffer per ticket, append every pre-built part, and return toString(). A list holds the generated ticket strings until the timed section ends, so the generated outputs are retained consistently by all three approaches.

The program performs three warm-up rounds per workload and approach, then five measured repetitions. System.nanoTime() measures the ticket-generation loop. The starting approach rotates on each repetition to reduce fixed-order bias. A checksum consumes the results after timing. Heap use is sampled immediately before and after each measured loop; a System.gc() request is made before each sample to reduce carry-over.

Execution environment: OpenJDK 21.0.12.1, OpenJDK 64-Bit Server VM. The supplied run used -Xms256m and -Xmx1g. Timing excludes input generation, CSV writing and checksum calculation.

5. Measured execution time

Workload	String (ms)	StringBuilder (ms)	StringBuffer (ms)
Short ticket (1 ticket x 6 fields)	0.077	0.080	0.062
High load (10,000 tickets x 50 fields)	40.695	5.025	5.515
Wide ticket (1,000 tickets x 200 fields)	34.550	2.808	2.825

The table reports the arithmetic mean of five measured runs, as required by the task. The raw individual run values are retained in task1_results.csv. The single-ticket times are all below one tenth of a millisecond and are close enough that small system and timer effects can dominate.

6. Estimated String-object creation

Workload	String intermediate objects	StringBuilder	StringBuffer
Short ticket (1 ticket x 6 fields)	5	0	0
High load (10,000 tickets x 50 fields)	490000	0	0
Wide ticket (1,000 tickets x 200 fields)	199000	0	0

For the String loop, one concatenation result is estimated per field per ticket. The final result is retained, while the preceding results are intermediate objects. Therefore, estimated intermediate objects are tickets x (fields - 1). For StringBuilder and StringBuffer, the estimate is zero intermediate String objects per ticket because append mutates a buffer; one final String is still created per ticket. These are source-level estimates, not allocation-profiler counts, and exclude buffer objects, internal character arrays and any JVM implementation details.

7. Heap snapshot observations

Workload	String (MB)	StringBuilder (MB)	StringBuffer (MB)
Short ticket (1 ticket x 6 fields)	0.21	0.26	0.26
High load (10,000 tickets x 50 fields)	74.55	32.50	32.50
Wide ticket (1,000 tickets x 200 fields)	61.92	13.00	13.00

These values are the average of (heap after generation - heap before generation), expressed in MiB. They are approximate snapshots. They are not peak memory, and they include the retained output list and any temporary objects that have not yet been collected at the time of the snapshot. The String approach shows larger deltas for the two larger workloads, consistent with its greater temporary allocation. Heap readings should not be treated as exact allocation counts.

8. Comparison of prediction and observation

The prediction is supported. For one six-field ticket, all three approaches complete in a very small time interval. This workload is too small to justify choosing a class on timing alone. At 10,000 tickets with 50 fields, the mean for String was 40.695 ms, compared with 5.025 ms for StringBuilder and 5.515 ms for StringBuffer. String concatenation was about 8.1 times the StringBuilder mean in this run. At 1,000 tickets with 200 fields, String averaged 34.550 ms, whereas StringBuilder and StringBuffer averaged 2.808 ms and 2.825 ms respectively.

The String measurements also showed greater run-to-run variation in the high-load workload. The observed high heap deltas and variation are consistent with the creation of many temporary strings and additional garbage-collection pressure. The exact values are JVM- and machine-dependent; the conclusion is based on the repeated measurements and the object-creation pattern, not a claim that these exact timings apply to every computer.

9. Approximate heap delta ranges

Workload / approach	String (MB)	StringBuilder (MB)	StringBuffer (MB)
Short ticket (1 ticket x 6 fields)	0.00 - 0.26	0.26 - 0.26	0.26 - 0.26
High load (10,000 tickets x 50 fields)	19.00 - 88.61	32.50 - 32.50	32.50 - 32.50
Wide ticket (1,000 tickets x 200 fields)	61.77 - 62.06	13.00 - 13.00	13.00 - 13.00

10. Design decision for the booking server

Per-ticket text

Use a local StringBuilder for each ticket. Every booking thread can create and use its own builder without sharing it with other threads. This avoids unnecessary synchronization and performed well in the larger workloads. Once the ticket has been converted to a String, the immutable result can be safely shared or transmitted without risk of later modification.

Shared booking log

For a simple shared mutable log, StringBuffer is the safer direct choice because its append methods are synchronized. However, synchronization of individual methods does not automatically make a sequence of several append calls one indivisible record. If a complete log record uses multiple calls, protect the whole record with one synchronized block on the same buffer. In a high-throughput server, a logging framework or a queue with a dedicated log-writer thread is usually a better design because it can reduce contention between booking threads.

11. Alternative concepts and complexity

The program's parts include their own separator prefixes, so the following alternatives join those parts with an empty delimiter while preserving the same ticket text:

```
String ticket = String.join("", parts);
```

```
StringJoiner joiner = new StringJoiner("");
for (String part : parts) { joiner.add(part); }
String ticket = joiner.toString();
```

```
String format = "%s".repeat(parts.length);
String ticket = String.format(format, (Object[]) parts);
```

String.join() and StringJoiner provide linear-time joining for the total characters copied. String.format() can also produce the output, but it must process a format pattern and arguments; for simple ticket assembly it has more formatting and varargs overhead than a builder. It is more useful when formatted widths, numeric formatting or locale-sensitive output are required.

Approach	Time complexity	Extra space / allocation
Repeated String +	O(n^2) character copying for bounded-size fields	Many intermediate Strings; cumulative temporary character allocation can be O(n^2)
StringBuilder	O(n) for bounded-size fields; amortized append	One mutable buffer plus final String; O(n) output-sized storage
StringBuffer	O(n) for bounded-size fields, with synchronization overhead	One synchronized mutable buffer plus final String; O(n) output-sized storage
String.join()	O(n) for bounded-size fields	Final String and joining implementation storage; O(n)
StringJoiner	O(n) for bounded-size fields	Joiner storage and final String; O(n)
String.format()	O(n) for fixed-size format and fields; higher constants	Formatter, arguments and final String; O(n)

Here n is the number of fields and each field is assumed to have bounded length. More precisely, if the final ticket contains L characters, StringBuilder, StringBuffer, String.join() and StringJoiner need O(L) character work. Repeated concatenation recopies the growing prefix after each field, giving O(n^2) total copied characters when field lengths are bounded. Its cumulative temporary allocation is also quadratic in that model, even though old intermediate objects may become eligible for garbage collection.

12. Conclusion

The experiment demonstrates that the choice of text-building class matters when a ticket contains many fields or when many tickets are generated. Repeated String concatenation is acceptable for a very small, fixed amount of text, but it creates many intermediate String objects in a loop. StringBuilder is the recommended per-ticket approach because it combines linear-time construction with no synchronization overhead when the builder is thread-confined. StringBuffer is suitable for a directly shared mutable log when synchronized access is required, subject to protecting complete multi-append records as one critical section.

13. Reproduction and submission evidence

The raw measurements are saved in task1_results.csv and the console output is saved in run_output.txt. To reproduce the experiment, open a terminal in the Task01 folder and execute:

```
javac TicketBenchmark.java
java -Xms256m -Xmx1g TicketBenchmark
```

The program creates a new CSV file in the current folder. Keep the raw measurement rows and capture a screenshot of the console output from the computer used for submission. The CSV and console output supplied with this report were generated in the stated OpenJDK environment. Because execution time and heap readings vary across systems, the final submitted evidence should be regenerated on the student's own computer when possible.

14. Measurement limitations

This is a controlled microbenchmark, not a full railway reservation server. It reuses pre-built parts and does not include database access, file writing, network transfer, or creation of unique passenger data for each ticket. The heap metric is an approximate before/after snapshot and cannot identify peak memory or exact allocation counts. Five repetitions provide repeatability evidence for this run, but longer runs and a profiler would be needed for precise allocation analysis. These limitations do not change the central distinction between immutable concatenation in a loop and appending to a mutable buffer.

End of report